

Ideation Phase

Brainstorm & Idea Prioritization Template

Date	3 October 2026
Team members	Priyanka Sri J, Abinaya R, Monika V
Project Name	Implementation of Client Script and UI Policy
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process that leads to problem solving. Prioritizing volume over value, out-of-the-box ideas are welcome and built upon, and all participants are encouraged to collaborate, helping each other develop a rich amount of creative solutions.

This template was used in our brainstorming session to shape ideas for the topic **Implementation of Client Script and UI Policy** on the ServiceNow platform, so that the team could unleash its imagination and start shaping concepts even when not sitting in the same room.

Reference: <https://www.mural.co/templates/brainstorm-and-idea-prioritization>

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Before you collaborate (10 minutes)	Define your problem statement (5 minutes)
A. Team gathering - Participants: Priyanka, Abinaya, Monika (2-8 people recommended). Share pre-work: ServiceNow forms, the Incident table, and basic JavaScript. B. Set the goal - Decide how to control form behaviour on the client side without unnecessary server round-trips. C. Learn the facilitation tools - Follow the key rules of brainstorming.	PROBLEM How might we control form behaviour (show/hide, mandatory, read-only and validation) on ServiceNow forms using Client Scripts and UI Policies, and decide when to use which one, so that forms are easy to use, accurate and fast?
Key rules of brainstorming - To run a smooth and productive session	

Step-2: Brainstorm, Idea Listing and Grouping

Brainstorm (10 minutes) - Each member wrote down ideas that address the problem statement.

Priyanka	Use a UI Policy to make 'Short description' mandatory	onChange Client Script to auto-fill Category from Caller	Hide 'Close notes' until State = Resolved	Show alert message on invalid input
Abinaya	UI Policy to make fields read-only after closure	onLoad script to set default Priority	Use g_form.setValue() to populate related fields	Check 'Run scripts' option for advanced UI Policy actions
Monika	onSubmit script to validate mandatory data	Use GlideAjax to fetch server data from the client	Show field messages with g_form.showFieldMsg()	Use UI Policy conditions instead of scripting
Abinaya	Use g_form.setDisplay() / setVisible() to hide fields	Test using different user roles	Use onCellEdit for list-level validation	Avoid using DOM manipulation in scripts

TIP: Sticky notes can be sketched or drawn to explain an idea visually.

Group ideas (20 minutes) - The team shared ideas, clustered similar notes and gave each cluster a sentence-like label.

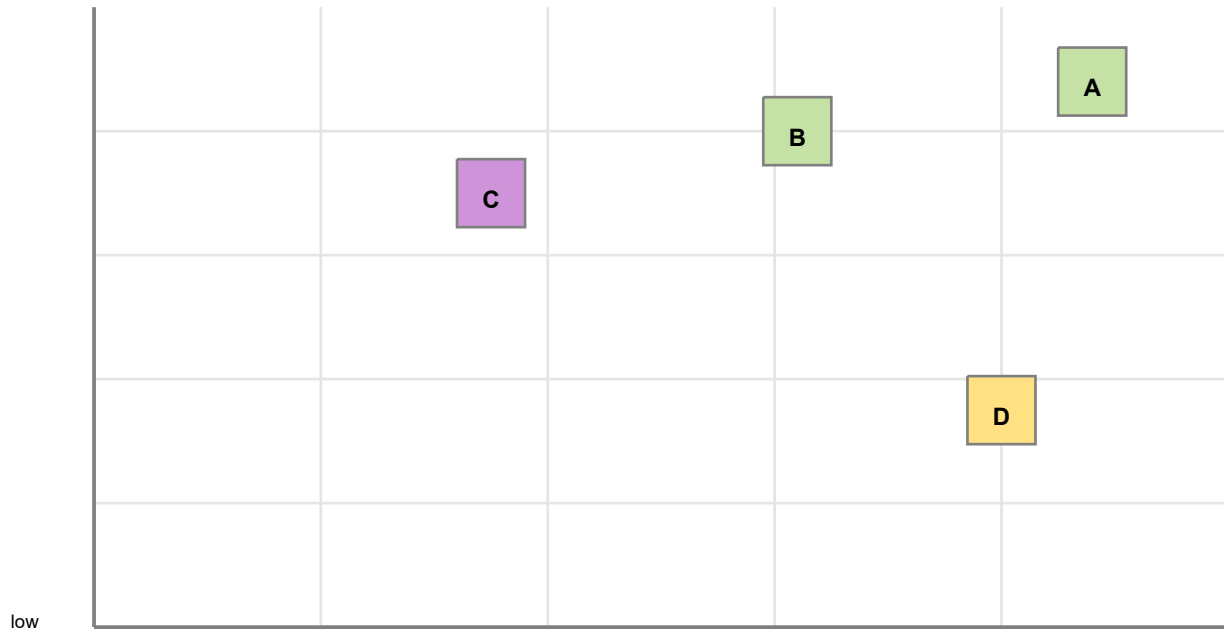
Theme / Cluster label	Ideas grouped
A. Use UI Policy for simple, no-code field control	Mandatory fields; read-only after closure; hide 'Close notes' until Resolved; conditions instead of scripting
B. Use Client Script for dynamic logic and validation	onChange auto-fill; onLoad defaults; onSubmit validation; alert and field messages; onCellEdit
C. Fetch and set data from the server efficiently	GlideAjax for server data; g_form.setValue() for related fields
D. Follow best practices and test thoroughly	Avoid DOM manipulation; test with different roles; prefer UI Policy over scripts where possible

TIP: Each cluster holds six or fewer sticky notes; larger clusters were broken into smaller sub-groups.

Step-3: Idea Prioritization

Prioritize (20 minutes) - The team placed each grouped idea on the grid to decide which ideas are important and which are feasible. **Importance:** if each task could get done without difficulty or cost, which would have the most positive impact? **Feasibility:** regardless of importance, which tasks are more feasible than others (cost, time, effort, complexity)?

Importance (high)



Feasibility (low to high)

Rank	Cluster	Importance	Feasibility	Decision
1	A. UI Policy for simple field control	High	High	Implement first
2	B. Client Script for dynamic logic and validation	High	High / Medium	Implement next
3	C. Fetch and set data using GlideAjax	Medium-High	Medium	Implement after A and B

4	D. Best practices and testing	Medium	High	Apply throughout
---	-------------------------------	--------	------	------------------

Outcome: The team will first apply UI Policies for simple, declarative form behaviour, then add Client Scripts only where logic, validation or server data is needed, keeping forms fast, consistent and easy to maintain.